

Лабораторная работа №2

Структуры данных

Доберштейн Алина Сергеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
5	Кортежи	9
6	Словари	10
7	Множества	11
8	Массивы	13
9	Задание 1	17
10	Задание 2	18
11	Задание 3	19
12	Задание 4	26
13	Задание 5	27
14	Задание 6	28
15	Выводы	29
	Список литературы	30

Список иллюстраций

5.1	Кортежи	9
5.2	Кортежи	9
6.1	Словари	10
6.2	Словари	10
7.1	Множества	11
7.2	Множества	11
7.3	Множества	12
8.1	Массивы	13
8.2	Массивы	13
8.3	Массивы	14
8.4	Массивы	14
8.5	Массивы	14
8.6	Массивы	15
8.7	Массивы	15
8.8	Массивы	15
8.9	Массивы	16
9.1	Задание 1	17
10.1	Задание 2	18
10.2	Задание 2	18
11.1	Задание 3.1	19
11.2	Задание 3.2	19
11.3	Задание 3.3	20
11.4	Задание 3.4	20
11.5	Задание 3.5	20
11.6	Задание 3.6	20
11.7	Задания 3.7, 3.8	20
11.8	Задание 3.9	21
11.9	Задание 3.10	21
11.10	Задание 3.11	21
11.11	Задание 3.12	21
11.12	Задание 3.13	22
11.13	Задание 3.14	22

11.14Задание 3.14	22
11.15Задание 3.14	23
11.16Задание 3.14	23
11.17Задание 3.14	24
11.18Задание 3.14	24
11.19Задание 3.14	24
11.20Задание 3.14	24
11.21Задание 3.14	25
11.22Задание 3.14	25
11.23Задание 3.14	25
12.1 Задание 4	26
13.1 Задание 5	27
13.2 Задание 5	27
14.1 Задание 6	28
14.2 Задание 6	28
14.3 Задание 6	28

1 Цель работы

Изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 2.2.
2. Выполните задания для самостоятельной работы.

3 Теоретическое введение

Рассмотрим несколько структур данных, реализованных в Julia. Несколько функций (методов), общих для всех структур данных: – `isempty()` — проверяет, пуста ли структура данных; – `length()` — возвращает длину структуры данных; – `in()` — проверяет принадлежность элемента к структуре; – `unique()` — возвращает коллекцию уникальных элементов структуры; – `reduce()` — свёртывает структуру данных в соответствии с заданным бинарным оператором; – `maximum()` (или `minimum()`) — возвращает наибольший (или наименьший) результат вызова функции для каждого элемента структуры данных.

4 Выполнение лабораторной работы

5 Кортежи

Кортеж(Tuple) — структура данных (контейнер) в виде неизменяемой индексируемой последовательности элементов какого-либо типа (элементы индексируются с единицы).

Повторила примеры из лабораторной работы с кортежами: (рис. 5.1-5.2)

```
[1]: # КОРТЕЖИ
[1]: ()
[2]: favouritelang = ("Julia", "Python", "R")
[2]: ("Julia", "Python", "R")
[3]: x1 = (1, 2, 3)
[3]: (1, 2, 3)
[4]: x2 = (1, 2.0, "tmp")
[4]: (1, 2.0, "tmp")
[8]: x3 = (a=2, b=1+2)
[8]: (a = 2, b = 3)
```

Рис. 5.1: Кортежи

```
[5]: length(x2)
[5]: 3
[6]: x2[1], x2[3]
[6]: (1, "tmp")
[7]: c = x1[1]*x1[3]
[7]: 4
[10]: x3.a, x3.b, x3[2]
[10]: (2, 3, 3)
[11]: in("tmp", x2)
[11]: true
[12]: 0 in x2
[12]: false
```

Рис. 5.2: Кортежи

6 Словари

Словарь — неупорядоченный набор связанных между собой по ключу данных.

Повторила примеры из лабораторной работы со словарями: (рис. 6.1-6.2)

```
[23]: # СЛОВАРИ
phonebook = Dict("Иванов И.И." => ("867-5309", "333-5544"), "Бухгалтерия" => "555-2368")

[23]: Dict{String, Any} with 2 entries:
  "Бухгалтерия" => "555-2368"
  "Иванов И.И." => ("867-5309", "333-5544")

[24]: keys(phonebook)

[24]: KeySet for a Dict{String, Any} with 2 entries. Keys:
  "Бухгалтерия"
  "Иванов И.И."

[25]: values(phonebook)

[25]: ValueIterator for a Dict{String, Any} with 2 entries. Values:
  "555-2368"
  ("867-5309", "333-5544")

[26]: pairs(phonebook)

[26]: Dict{String, Any} with 2 entries:
  "Бухгалтерия" => "555-2368"
  "Иванов И.И." => ("867-5309", "333-5544")
```

Рис. 6.1: Словари

```
[27]: haskey(phonebook, "Иванов И.И.")

[27]: true

[28]: phonebook["Сидоров П.С."] = "555-3344"

[28]: "555-3344"

[29]: pop!(phonebook, "Иванов И.И.")

[29]: ("867-5309", "333-5544")

[30]: phonebook

[30]: Dict{String, Any} with 2 entries:
  "Сидоров П.С." => "555-3344"
  "Бухгалтерия" => "555-2368"

[31]: a = Dict{"foo" => 0.0, "bar" => 42.0};
      b = Dict{"baz" => 17, "bar" => 13.0}

[31]: Dict{String, Real} with 2 entries:
  "bar" => 13.0
  "baz" => 17

[32]: merge(a, b), merge(b, a)

[32]: (Dict{String, Real}{"bar" => 13.0, "baz" => 17, "foo" => 0.0}, Dict{String, Real}{"bar" => 42.0, "baz" => 17, "foo" => 0.0})
```

Рис. 6.2: Словари

7 Множества

Множество, как структура данных в Julia, соответствует множеству, как математическому объекту, то есть является неупорядоченной совокупностью элементов какого-либо типа.

Повторила примеры из лабораторной работы с множествами: (рис. 7.1-7.3)

```
[33]: # Множества
      A = Set{Int64}([1, 2, 3, 4, 5])

[33]: Set{Int64} with 5 elements:
      5
      4
      2
      3
      1

[34]: B = Set{Char}("abracadabra")

[34]: Set{Char} with 5 elements:
      'a'
      'd'
      'r'
      'k'
      'b'

[35]: S1 = Set{Int64}([1, 2]);
      S2 = Set{Int64}([3, 4])

[35]: Set{Int64} with 2 elements:
      4
      3

[36]: issetequal(S1, S2)

[36]: false
```

Рис. 7.1: Множества

```
[37]: S3 = Set{Int64}([1, 2, 2, 3, 1, 2, 3, 2, 1]);
      S4 = Set{Int64}([2, 3, 1]);

[38]: issetequal(S3, S4)

[38]: true

[39]: C = union(S1, S2)

[39]: Set{Int64} with 4 elements:
      4
      2
      3
      1

[40]: D = intersect(S1, S3)

[40]: Set{Int64} with 2 elements:
      2
      1

[41]: E = setdiff(S3, S1)

[41]: Set{Int64} with 1 element:
      3

[42]: issubset(S1, S4)

[42]: true
```

Рис. 7.2: Множества

```

[43]: push!(S4, 99)
[43]: Set{Int64} with 4 elements:
      2
      99
      3
      1
[44]: S4
[44]: Set{Int64} with 4 elements:
      2
      99
      3
      1
[45]: pop!(S4)
[45]: 2
[46]: S4
[46]: Set{Int64} with 3 elements:
      99
      3
      1

```

Рис. 7.3: Множества

8 Массивы

Массив — коллекция упорядоченных элементов, размещённая в многомерной сетке. Векторы и матрицы являются частными случаями массивов.

Повторила примеры из лабораторной работы с массивами: (рис. 8.1-8.9)

```
[47]: # МАССИВЫ
      empty_array_1 = []
      empty_array_2 = (Int64){}
      empty_array_3 = (Float64){}

[47]: Float64[]

[48]: a = [1, 2, 3]

[48]: 3-element Vector{Int64}:
      1
      2
      3

[49]: b = [1 2 3]

[49]: 1x3 Matrix{Int64}:
      1  2  3

[50]: A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
      B = [[1 2 3]; [4 5 6]; [7 8 9]]

[50]: 3x3 Matrix{Int64}:
      1  2  3
      4  5  6
      7  8  9
```

Рис. 8.1: Массивы

```
[51]: c = rand(1, 8)

[51]: 1x8 Matrix{Float64}:
      0.737116  0.148382  0.395577  0.823045  - 0.426358  0.494639  0.285114

[52]: c = rand(2, 3)

[52]: 2x3 Matrix{Float64}:
      0.368817  0.832425  0.750252
      0.185818  0.969841  0.695554

[53]: D = rand(4, 3, 2)

[53]: 4x3x2 Array{Float64, 3}:
     [:, :, 1] =
      0.993253  0.127375  0.508729
      0.558835  0.447448  0.539004
      0.517738  0.272906  0.788247
      0.152752  0.202545  0.792362

     [:, :, 2] =
      0.15834  0.81987  0.428634
      0.776869  0.624933  0.663593
      0.728429  0.348095  0.572788
      0.598728  0.926992  0.885683
```

Рис. 8.2: Массивы

```
[54]: roots = [sqrt(i) for i in 1:10]

[54]: 10-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795

[55]: an_1 = [3*i^2 for i in 1:2:9]

[55]: 5-element Vector{Int64}:
 3
 27
 75
 147
 243

[56]: an_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]

[56]: 4-element Vector{Int64}:
 1
 9
 49
 81
```

Рис. 8.3: Массивы

```
[57]: ones(5)

[57]: 5-element Vector{Float64}:
 1.0
 1.0
 1.0
 1.0
 1.0

[58]: ones(2, 3)

[58]: 2x3 Matrix{Float64}:
 1.0 1.0 1.0
 1.0 1.0 1.0

[59]: zeros(4)

[59]: 4-element Vector{Float64}:
 0.0
 0.0
 0.0
 0.0

[60]: fill(3.5, (3, 2))

[60]: 3x2 Matrix{Float64}:
 3.5 3.5
 3.5 3.5
 3.5 3.5
```

Рис. 8.4: Массивы

```
[61]: repeat([1,2],3,3)

[61]: 6x3 Matrix{Int64}:
 1 1 1
 2 2 2
 1 1 1
 2 2 2
 1 1 1
 2 2 2

[62]: a = collect(1:12)

[62]: 12-element Vector{Int64}:
 1
 2
 3
 4
 5
 6
 7
 8
 9
 10
 11
 12

[63]: b = reshape(a, (2, 6))

[63]: 2x6 Matrix{Int64}:
 1 3 5 7 9 11
 2 4 6 8 10 12
```

Рис. 8.5: Массивы

```
[64]: b'
```

```
[64]: 6x2 adjoint(::Matrix{Int64}) with eltype Int64:
      1  2
      3  4
      5  6
      7  8
      9 10
     11 12
```

```
[66]: c = transpose(b)
```

```
[66]: 6x2 transpose(::Matrix{Int64}) with eltype Int64:
      1  2
      3  4
      5  6
      7  8
      9 10
     11 12
```

```
[68]: ar = rand(10:20, 10, 5)
```

```
[68]: 10x5 Matrix{Int64}:
      16 14 18 12 16
      18 13 17 20 19
      16 19 17 15 18
      17 19 12 10 17
      14 16 17 14 12
      13 20 17 16 12
      19 16 12 13 17
      16 12 16 13 13
      18 10 14 16 19
      10 10 15 11 14
```

Рис. 8.6: Массивы

```
[69]: ar[:, 2]
```

```
[69]: 10-element Vector{Int64}:
      14
      13
      19
      19
      16
      20
      16
      12
      10
      10
```

```
[71]: ar[:, [2,5]]
```

```
[71]: 10x2 Matrix{Int64}:
      14 16
      13 18
      19 18
      19 17
      16 12
      20 12
      16 17
      12 13
      10 19
      10 14
```

Рис. 8.7: Массивы

```
[72]: ar[[2,4,6], [1,5]]
```

```
[72]: 3x2 Matrix{Int64}:
      18 18
      17 17
      13 12
```

```
[73]: ar[1,3:end]
```

```
[73]: 3-element Vector{Int64}:
      18
      12
      16
```

Рис. 8.8: Массивы

```
[74]: sort(ar,dims=1)

[74]: 10x5 Matrix{Int64}:
      10  10  12  10  12
      13  10  12  11  12
      14  12  14  12  13
      16  13  15  13  14
      16  14  16  13  16
      16  16  17  14  17
      17  16  17  15  17
      18  19  17  16  18
      18  19  17  16  18
      19  20  18  20  19

[75]: sort(ar,dims=2)

[75]: 10x5 Matrix{Int64}:
      12  14  16  16  18
      13  17  18  18  20
      15  16  17  18  19
      10  12  17  17  19
      12  14  14  16  17
      12  13  16  17  20
      12  13  16  17  19
      12  13  13  16  16
      10  14  16  18  19
      10  10  11  14  15
```

Рис. 8.9: Массивы

9 Задание 1

Выполнила задание для самостоятельной работы №1 (рис. 9.1)

1. Даны множества: $A = \{0, 3, 4, 9\}$, $B = \{1, 3, 4, 7\}$, $C = \{0, 1, 2, 4, 7, 8, 9\}$. Найти $P = A \cap B \cup A \cap B \cup A \cap C \cup B \cap C$.

```
[80]: A = Set([0, 3, 4, 9])
      B = Set([1, 3, 4, 7])
      C = Set([0, 1, 2, 4, 7, 8, 9]);

[81]: P = union(intersect(A, B), intersect(A, B), intersect(A, C), intersect(B, C))

[81]: Set{Int64} with 6 elements:
      0
      4
      7
      9
      3
      1
```

Рис. 9.1: Задание 1

10 Задание 2

Выполнила задание для самостоятельной работы №2 (рис. 10.1-10.2)

2. Приведите свои примеры с выполнением операций над множествами элементов разных типов.

```
[91]: Set1 = Set([1, 2, 3, "Hello", "World"]);  
[92]: print(Set1)  
Set{Any{2, "Hello", "World", 3, 1}}  
[93]: in("World", Set1)  
[93]: true  
[94]: push!(Set1, "JuliaLang")  
[94]: Set{Any} with 6 elements:  
2  
"Hello"  
"World"  
"JuliaLang"  
3  
1
```

Рис. 10.1: Задание 2

```
[95]: pop!(Set1, 3)  
[95]: 3  
[97]: Set1  
[97]: Set{Any} with 5 elements:  
2  
"Hello"  
"World"  
"JuliaLang"  
1
```

Рис. 10.2: Задание 2

11 Задание 3

Выполнила задание для самостоятельной работы №3 (рис. 11.1-11.23)

3. Создайте разными способами:

3.1) массив $(1, 2, 3, \dots, N-1, N)$, N выберите больше 20;

```
98]: N = 22
m = [i for i in 1:N]

98]: 22-element Vector{Int64}:
 1
 2
 3
 4
 5
 6
 7
 8
 9
10
11
12
13
14
15
16
17
18
19
20
21
22
```

Рис. 11.1: Задание 3.1

3.2) массив $(N, N-1, \dots, 2, 1)$, N выберите больше 20;

```
112]: rev_m = reverse([i for i in 1:N])

112]: 22-element Vector{Int64}:
22
21
20
19
18
17
16
15
14
13
12
11
10
 9
 8
 7
 6
 5
 4
 3
 2
 1
```

Рис. 11.2: Задание 3.2

```
print(vcat(m[1:21], rev_m))
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

3.4) массив с именем `tmp` вида $(4, 6, 3)$;

```
28]: tmp = [4 6 3]
      tmp

28]: 1x3 Matrix{Int64}:
      4  6  3
```

3.5) массив, в котором первый элемент массива tmp повторяется 10 раз;

```
[29]: tmp_1 = fill(tmp[1], 10)
[29]: 10-element Vector{Int64}:
      4
      4
      4
      4
      4
      4
      4
      4
      4
      4
```

3.6) массив, в котором все элементы массива tmp повторяются 10 раз;

[illegible]

3.7) массив, в котором первый элемент массива `tmp` встречается 11 раз, второй элемент — 10 раз, третий элемент — 10 раз;

```
48) tmp_3 = print(vcat(fill(tmp, [11, 10, 10])...))
```

3.8) массив, в котором первый элемент массива tmp встречается 10 раз подряд, второй элемент – 20 раз подряд, третий элемент – 30 раз подряд;

```
49) tmp_4 = print(vcat(fill.(tmp, [10, 20, 30]),...))
```

20

```
[150]: ar_1 = [2*tmp[i] for i in 1:1:3]

[150]: 3-element Vector{Int64}:
       16
       64
       8

[151]: ar_2 = print(vcat(fill(ar_1, [1, 1, 4])...))

[16, 64, 8, 8, 8, 8]

[154]: n = 0
       for i in ar_1
           if '6' in string(i)
               n += 1
           end
       end

[155]: print(n)

2
```

```
64]: f(x) = exp(x)*cos(x)
64]: f (generic function with 1 method)
65]: x = [f(x) for x in 3:0.1:6];
72]: using Statistics
    res = f.(x)
    mean(res)
72]: -3.199908837098971e166
```

```

79] x = 0.1
    y = 0.2;

80] v = []
    for j in 1:3:34
        i = j + 2
        push!(v, xi, yj)
    end

81] print(v)

Any{Array{Float64,2}} = Array{Float64,2} = 0.00000000000000000002, 0.2, 1.60000000000000000004e-6, 0.0016000000000000000003, 1.60000000000000000005e-9, 1.28000000000000000005e-5, 1.00000000000000000006e-12, 1.024
00000000000000000009e-15, 0.19200000000000000005e-10, 1.00000000000000000001e-18, 6.55360000000000000005e-12, 1.000000000000000000012e-21, 5.24288000000000000005
6e-14, 1.000000000000000000014e-24, 4.19430400000000000005e-16, 1.0000000000000000000015e-27, 3.35544300000000000004e-18, 1.0000000000000000000017e-30, 2.684315450000000000e-20, 1.00
0000000000000000000018e-33, 2.14748364800000000005e-22, 1.00000000000000000002e-36, 1.7179869184000003e-24]

```

```

v = []
M = 25
for i in 1:M
    push!(v, (2^i)/i!)
end

print(v)

Any{2, 0, 2, 0.6666666666666665, 0.4, 6.4, 10.666666666666666, 18.285714285714285, 32.0, 56.888888888888886, 102.4, 186.18181818181818, 341.3333333333333, 638.1538461538462, 1170.2857142857142, 2184.5333333333333, 4096.0, 7710.117647058823, 14563.555555555555, 27594.10526315789, 52428.8, 99864.38095238095, 190650.18181818182, 364722.0869565217, 699050.6666666666, 1.3421728e6}

```

3.13) вектор вида ("fn1", "fn2", ..., "fnN"), $N = 30$;

```
86]: v = []
      N = 30
      for i in 1:1:N
          push!(v, "fn%i")
      end
      print(v)

Any["fn1", "fn2", "fn3", "fn4", "fn5", "fn6", "fn7", "fn8", "fn9", "fn10", "fn11", "fn12", "fn13", "fn14", "fn15", "fn16", "fn17", "fn18", "fn19", "fn20", "fn21", "fn22", "fn23", "fn24", "fn25", "fn26", "fn27", "fn28", "fn29", "fn30"]
```

Рис. 11.12: Задание 3.13

3.14) векторы $x = (x_1, x_2, \dots, x_n)$ и $y = (y_1, y_2, \dots, y_n)$ целочисленного типа длины $n = 250$ как случайные выборки из совокупности $0, 1, \dots, 999$; на его основе:
– сформируйте вектор $(y_2 - x_1, \dots, y_n - x_{n-1})$;

```
228]: n = 250
      x = rand(0:999, n)
      y = rand(0:999, n)
      length(x), length(y)

228]: (250, 250)
```

Рис. 11.13: Задание 3.14

```
:29]: v = []

      for i in 1:1:249
          j = i + 1
          push!(v, y[j]-x[i])
      end

      v

:29]: 249-element Vector{Any}:
      -113
       57
      592
       73
      -453
      -657
       216
      -86
      -165
      -55
       269
      -486
      390
       :
      215
      -358
      542
      -353
       65
       54
      -771
      567
      -650
      -139
      -70
      -900
```

Рис. 11.14: Задание 3.14

– сформируйте вектор $(x_1 + 2x_2 - x_3, x_2 + 2x_3 - x_4, \dots, x_{n-2} + 2x_{n-1} - x_n)$;

```
[230]: v = []

for i in 1:1:248
    j = i + 1
    k = i + 2
    push!(v, x[i]*2*x[j]-x[k])
end

v

[230]: 248-element Vector{Any}:
 1921
 367
 1300
 1696
 1740
 1001
 999
 1145
 1543
 1643
 1431
 410
 -439
 1
 132
 1600
 958
 1328
 1758
 691
 2159
 5
 1754
```

Рис. 11.15: Задание 3.14

– сформируйте вектор $\left(\frac{\sin(y_1)}{\cos(x_2)}, \frac{\sin(y_2)}{\cos(x_3)}, \dots, \frac{\sin(y_{n-1})}{\cos(x_n)} \right)$;

```
[231]: v = []

for j in 1:1:249
    i = j + 1
    push!(v, sin(y[j])/cos(x[i]))
end

v

[231]: 249-element Vector{Any}:
 0.5664614396518219
 -0.997949116199616
 -0.45937388275134255
 -44.10094601199591
 1.7500454236178697
 -1.327847471078773
 -0.8140359563178885
 0.1490732683833474
 -1.375280314031514
 35.82137204161159
 1.1196207455483171
 0.421254403755241
 -2.372866420480721
 1
 -0.6568423089458847
 1.69560814240803
 -0.5400351780071951
 4.554629413683212
 -0.31572189099295755
 -0.7619693729957793
 -0.09657217563663885
 1.2448183700229558
```

Рис. 11.16: Задание 3.14

– вычислите $\sum_{i=1}^{n-1} \frac{e^{-x_{i+1}}}{x_i + 10}$;

```
232]: res = 0

for i in 1:(n-1)
    sum = exp(-x[i+1])/(x[i]+10)
    res = res + sum
end

res

233]: 4.248848643745491e-8
```

Рис. 11.17: Задание 3.14

– выберите элементы вектора y , значения которых больше 600, и выведите на экран; определите индексы этих элементов;

```
238]: v = []

for i in y
    if i > 600
        push!(v, i)
    end
end

println("Элементы:"), println(v), println("Индексы:"), print(findall(y .-> 600))

Элементы:
Any{875, 679, 903, 950, 663, 716, 924, 996, 743, 947, 718, 664, 900, 778, 715, 826, 727, 802, 651, 645, 925, 933, 609, 809, 659, 904, 869, 886, 704, 926, 743, 638, 633, 912, 927, 803, 675, 844, 879, 764, 942, 919, 751, 847, 888, 763, 903, 874, 787, 938, 981, 764, 826, 629, 656, 797, 643, 816, 988, 632, 94, 4, 987, 795, 868, 853, 740, 710, 811, 765, 746, 713, 791, 815, 673, 743, 880, 947, 699, 895, 714, 684, 603, 924, 713, 791, 998, 603, 978, 707, 683, 731, 606, 743, 824, 641, 966, 847, 758, 681, 866, 883, 868, 960, 634, 872, 741, 718, 962, 761}

Индексы:
[2, 3, 4, 5, 8, 11, 12, 16, 21, 23, 26, 27, 28, 29, 30, 31, 33, 41, 42, 44, 45, 46, 49, 50, 51, 54, 55, 56, 57, 58, 60, 62, 65, 67, 69, 73, 74, 75, 76, 7, 7, 81, 83, 86, 89, 90, 91, 96, 98, 99, 102, 103, 104, 106, 107, 110, 114, 116, 124, 127, 129, 130, 132, 135, 139, 140, 141, 142, 143, 144, 145, 151, 152, 153, 155, 157, 159, 161, 163, 164, 166, 170, 171, 172, 173, 175, 176, 177, 179, 180, 181, 184, 185, 186, 188, 195, 196, 199, 201, 204, 208, 216, 222, 22, 5, 229, 230, 235, 236, 241, 243]
```

Рис. 11.18: Задание 3.14

– определите значения вектора x , соответствующие значениям вектора y , значения которых больше 600 (под соответствием понимается расположение на аналогичных индексных позициях);

```
240]: v = []

for i in 1:n
    if(y[i] > 600)
        push!(v, x[i])
    end
end

print(v)

Any{622, 311, 877, 765, 604, 655, 438, 996, 30, 763, 319, 858, 584, 181, 92, 185, 309, 926, 850, 105, 903, 781, 494, 663, 342, 747, 116, 791, 66, 683, 43, 1, 246, 977, 924, 906, 433, 181, 520, 463, 478, 203, 40, 365, 151, 841, 824, 704, 727, 817, 290, 557, 796, 496, 508, 900, 115, 216, 923, 336, 440, 773, 9, 11, 765, 305, 711, 780, 495, 823, 898, 857, 990, 797, 919, 91, 636, 942, 744, 623, 528, 915, 841, 964, 700, 647, 251, 245, 780, 872, 39, 269, 89, 289, 33, 6, 56, 524, 419, 499, 79, 455, 531, 259, 11, 388, 979, 799, 725, 353, 802, 436]
```

Рис. 11.19: Задание 3.14

– сформируйте вектор $(|x_1 - \bar{x}|^{\frac{1}{2}}, |x_2 - \bar{x}|^{\frac{1}{2}}, \dots, |x_n - \bar{x}|^{\frac{1}{2}})$, где $\bar{x}$ обозначает среднее значение вектора $x = (x_1, x_2, \dots, x_n)$;

```
247]: v = [(abs(i-mean(x)))^(1/2) for i in x]
print(v)

[21.609627484803672, 10.048681505550865, 14.492204801202611, 18.867326254665763, 15.619731111642094, 13.783178153096621, 8.60372012562008, 9.109116312793, 464, 11.617917195717157, 15.818629335987862, 11.574880214258559, 9.1175965506075, 20.518869364562953, 17.11794308175376, 21.770908971845515, 21.79394111, 2080985, 11.790843905316038, 14.105885296570364, 5.569919209467944, 19.72369133808375, 22.15906135196364, 15.747253728825227, 15.5555777763004, 16.6726, 12272826356, 11.400701732788205, 14.213514695528337, 18.356906057394312, 7.935741931287836, 18.439739694475083, 20.712894534564693, 18.330957421804243, 1, 5.96947087413982, 14.56104391862067, 5.198461310811114, 4.356145084819834, 13.893307741499143, 17.606135294266032, 13.227849409484522, 18.78765552164505, 7, 4.89652938314476, 20.124015503869998, 18.13769553735597, 22.068620255919942, 20.396666394291003, 19.544206302630804, 16.123771271014732, 21.955044078, 318764, 20.6403468342615, 5.198461310811114, 11.915368227629392, 13.379585852308541, 17.74891545982458, 20.56638031296008, 15.8324081290536, 20.125208073, 45852, 16.430046441218106, 21.331291568060374, 12.726979217394833, 16.64255955135326, 9.488097807253043, 7.747515730864959, 16.583847563216445, 7.6141972, 65634769, 20.34759708653313, 21.35359454518138, 16.674051699173066, 20.07426212840711, 21.56441513234245, 19.620085284187497, 22.42819653917809, 4.79833, 3043880969, 21.78119653385973, 9.38211063738409, 18.439739694475083, 1.0119288512538818, 7.617348620090851, 6.559268251870783, 18.02709072479528, 19.28, 667934093373, 17.7207212312907314, 17.83322741401567, 9.0013321458253, 21.932259345539393, 16.5536702878040, 19.206679340093373, 12.400956728769818, 20.44, 4461352650013, 13.748599928720015, 19.236007901849074, 17.887872987026714, 17.406205789898995, 22.226565066986027, 8.186818673941668, 13.415513407991511, 21.541216307340035, 13.526862163857514, 22.539387746786733, 14.351863990436922, 17.20395303411399, 17.747563213016033, 9.644895022756858, 15.199473675098, 096, 5.997999666555095, 16.582400308761095, 7.617348620090851, 5.002399424276314, 3.6088779419647876, 16.216534771645883, 3.996998874154558, 19.46730892, 5576883, 12.885030873694847, 5.918107805789524, 5.918107805789524, 20.150037220809295, 10.629016887746486, 17.464936301857612, 19.7731305788747, 17.775, 7157060877677 3, 870604046530856 22 7401740174650477 15 1465423114895777 17 600157046766017 15 030410470500005 20 040513014173007 17 0370103620767008
```

Рис. 11.20: Задание 3.14


```
[249]: res = ([i for i in y if abs(maximum(y)-i)<=200])
length(res)
```

```
[249]: 53
```

```
253]: chet = count(true for i in x if i%2==0)
      nechet = count(true for i in x if i%2!=0)
      print("Четных: ", chet, ", нечетных: ", nechet)

Четных: 118, нечетных: 132
```

```
[254]: krat = count(true for i in x if i%7==0)
```

```
[254]: 33
```

- отсортируйте элементы вектора x в порядке возрастания элементов вектора y ;

```
[58]: sorted = sortperm(y)
      sorted_x = x[sorted]
      sorted_x

[59]: 250-element Vector{Int64}:
      477
      326
      919
      200
      831
      461
      592
      873
      100
      864
      494
      837
      447
      1
      763
      744
      765
      388
      802
      419
      872
      557
      911
      336
      996
      245
```

- выведите элементы вектора x , которые входят в десятку наибольших (top-10)?

```
[61]: x_sorted = sort(x)
      top = reverse(x_sorted)
      top_10 = top[1:10]
      top_10
```

```
[61]: 10-element Vector{Int64}:
      996
      995
      992
      990
      989
      988
      979
      977
      964
      944
```

```
x_unique = unique(x)
print(x_unique)
```

[988, 621, 111, 877, 765, 711, 447, 604, 656, 771, 655, 438, 100, 228, 995, 996, 382, 940, 132, 30, 769, 763, 769, 651, 319, 858, 584, 584, 461, 461, 246, 579, 10, 266, 309, 494, 540, 328, 831, 606, 674, 545, 926, 850, 34, 105, 983, 781, 39, 59, 663, 342, 206, 944, 747, 116, 791, 66, 683, 798, 431, 461, 246, 579, 10, 7, 977, 243, 924, 56, 906, 18, 498, 64, 433, 520, 463, 478, 846, 893, 207, 203, 440, 40, 247, 365, 939, 332, 151, 841, 824, 2, 454, 107, 57, 784, 13, 72, 817, 836, 428, 290, 557, 796, 406, 508, 784, 537, 900, 555, 486, 484, 115, 634, 216, 912, 837, 506, 26, 201, 267, 923, 688, 457, 436, 561, 733, 280, 9, 10, 323, 323, 454, 305, 708, 406, 823, 809, 857, 62, 345, 329, 180, 377, 908, 707, 519, 571, 91, 866, 636, 924, 121, 764, 222, 334, 370, 915, 5, 912, 209, 904, 780, 447, 223, 251, 245, 581, 270, 320, 126, 829, 89, 289, 992, 418, 521, 724, 708, 526, 419, 159, 409, 567, 79, 265, 379, 455, 929, 9, 531, 140, 81, 259, 221, 601, 619, 415, 11, 381, 227, 388, 224, 146, 079, 864, 768, 438, 134, 725, 353, 674, 920, 420, 802, 436, 31, 934, 145, 326, 929]

25

12 Задание 4

Выполнила задание для самостоятельной работы №4 (рис. 12.1)

4. Создайте массив `squares`, в котором будут храниться квадраты всех целых чисел от 1 до 100.

```
265]: squares = [x**2 for x in range(1,101)]
      print(squares)

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 961, 1024, 1089, 1156, 1225, 1296, 1369, 1444, 1521, 1600, 1681, 1764, 1849, 1936, 2025, 2116, 2209, 2304, 2401, 2500, 2601, 2704, 2809, 2916, 3025, 3136, 3249, 3364, 3481, 3600, 3721, 3844, 3969, 4096, 4225, 4356, 4489, 4624, 4761, 4900, 5041, 5184, 5329, 5476, 5625, 5776, 5929, 6084, 6241, 6400, 6561, 6724, 6889, 7056, 7225, 7396, 7569, 7744, 7921, 8100, 8281, 8464, 8649, 8836, 9025, 9216, 9409, 9604, 9801, 10000]
```

Рис. 12.1: Задание 4

13 Задание 5

Выполнила задание для самостоятельной работы №5 (рис. 13.1-13.2)

5. Подключите пакет Primes (функции для вычисления простых чисел). Сгенерируйте массив `murprimes`, в котором будут храниться первые 168 простых чисел. Определите 89-е наименьшее простое число. Получите срез массива с 89-го до 99-го элемента включительно, содержащий наименьшие простые числа.

```
[268]: import Pkg; Pkg.add("Primes")

Updating registry at "C:\Users\MI\julia\registries\General.toml"
Resolving package versions...
Updating "C:\Users\MI\julia\environments\v1.11\Project.toml"
[27ebfcd6] + Primes v0.5.7
No Changes to "C:\Users\MI\julia\environments\v1.11\Manifest.toml"
Precompiling project...
4167.7 ms ✓ Plots + IJuliaExt
1 dependency successfully precompiled in 10 seconds. 514 already precompiled.

[269]: using Primes

prime_m = primes(prime(168))
print(prime_m)

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643, 647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829, 839, 853, 857, 859, 863, 877, 881, 883, 887, 907, 911, 919, 929, 937, 941, 947, 953, 967, 971, 977, 983, 991, 997]
```

Рис. 13.1: Задание 5

```
[270]: prime_m[89]

461

[271]: prime_m[89:99]

11-element Vector{Int64}:
 461
 463
 467
 479
 487
 491
 499
 503
 509
 521
 523
```

Рис. 13.2: Задание 5

14 Задание 6

Выполнила задание для самостоятельной работы №6 (рис. 14.1-14.3)

6. Вычислите следующие выражения:

$$6.1) \sum_{i=10}^{100} (i^3 + 4i^2);$$

```
[273]: res = 0
for i in 10:1:100
    res = res + (i^3 + 4*i^2)
end
res
[273]: 26852735
```

Рис. 14.1: Задание 6

$$6.2) \sum_{i=1}^M \left(\frac{2^i}{i} + \frac{3^i}{i^2} \right), M = 25;$$

```
275]: res = 0
M = 25
for i in 1:1:M
    res = res + ((2^i)/i + (3^i)/(i^2))
end
res
275]: 2.1291704368143802e9
```

Рис. 14.2: Задание 6

$$6.3) 1 + \frac{2}{3} + \left(\frac{2}{3} \frac{4}{5} \right) + \left(\frac{2}{3} \frac{4}{5} \frac{6}{7} \right) + \dots + \left(\frac{2}{3} \frac{4}{5} \dots \frac{38}{39} \right).$$

```
276]: res = 1
tmp = 1
for i in 2:2:38
    tmp = tmp*(i/(i+1))
    res = res + tmp
end
res
276]: 6.976346137897618
```

Рис. 14.3: Задание 6

15 Выводы

В результате выполнения данной лабораторной работы я изучила несколько структур данных, реализованных в Julia, научилась применять их и операции над ними для решения задач.

Список литературы